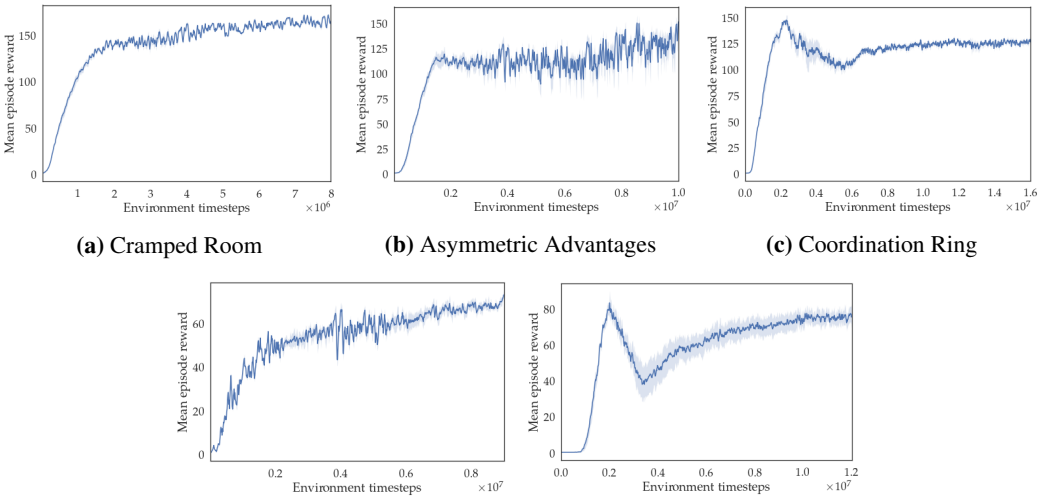


(a) Cramped Room **(b)** Asymmetric Advantages **(c)** Coordination Ring

(d) Forced Coordination **(e)** Counter Circuit

Figure 10: PPO$_{H_{Proxy}}$ average episode rewards on each layout during training over 400 horizon timesteps, when pairing the agent with itself or with H$_{Proxy}$ in proportion to the current self-play annealing.

entropy coefficient (initialized $= 0.5$), and value function coefficient (initialized $= 0.1$). At each PBT step, each parameter had a 33% chance of being mutated by either a factor of 0.75 or 1.25 (and clipped to the closest integer if necessary). For the lambda parameter, we mutate by $\pm\frac{\epsilon}{2}$ where ϵ is the distance to the closest of 0 or 1, to ensure that it will not go out of bounds.

As for the other DRL algorithms, we use reward shaping and anneal it linearly throughout training, and evaluate PBT reporting means and standard errors over 5 seeds. Hyperparameters and training reward curves are reported respectively in Table 4 and Figure 11.

PBT hyperparameters					
Parameter	Cramped Rm.	Asym. Adv.	Coord. Ring	Forced Coord.	Counter Circ.
Learning Rate	2e-3	8e-4	8e-4	3e-3	1e-3
Rew. shaping horizon	3e6	5e6	4e6	7e6	4e6
Env. steps per agent	8e6	1.1e7	5e6	8e6	6e6
# Minibatches	10	10	10	10	10
Minibatch size	2000	2000	2000	2000	2000
PPO iter. timesteps	40000	40000	40000	40000	40000

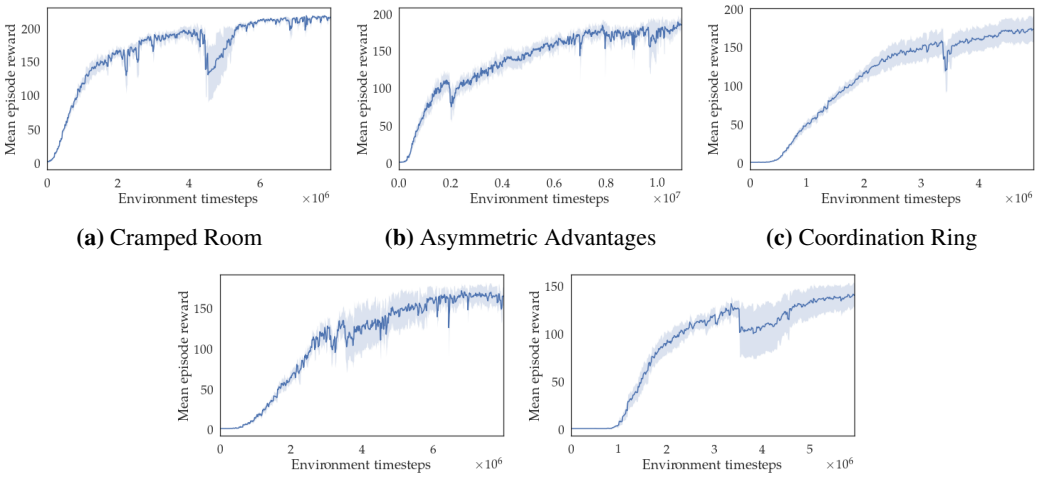
Table 4: Hyperparameters for PBT, across the 5 layouts. *PPO iteration timesteps* refers to the length in environment timesteps for each agent pairing training. For simulation, we used 50 parallel environments. The mutation parameters were equal across all layouts. For further description of the parameters, see Table 3.

E Near-optimal joint planner

As mentioned in 4.3, to perform optimal planning we pre-compute optimal joint motion plans for every possible starting and desired goal location for each agent. This enables us to quickly query the cost of each motion plan when performing A^* search.

We then define the high-level actions: "get an onion", "serve the dish", etc, and map each joint high-level action onto specific joint motion plans. We then use A^* search to find the optimal joint plan in this high-level action space.

This planner does make some assumptions for computational reasons, making it near-optimal instead of optimal. In order to reduce the number of required joint motion plans by a factor of 16, we only consider position-states for the players, and not their orientations. This adds the possibility of wasting one timestep when executing certain motion plans, in order to get into the correct orientation. We



(a) Cramped Room **(b)** Asymmetric Advantages **(c)** Coordination Ring

(d) Forced Coordination **(e)** Counter Circuit

Figure 11: Average episode rewards on each layout during training for PBT agents when paired with each other, averaged across all agents in the population.

have added a set of additional conditions to check for such a case and reduce the impact of such an approximation, but they are not general.

Another limitation of the planning logic is that counters are not considered when selecting high level action, as it would increase the runtime of A^* by a large amount (since the number of actions available when holding an item would greatly increase). This is not of large importance in the two layouts we run planning experiments on. The use of counters in such scenarios is also minimal in human gameplay.

Another approximation made by the planner is only considering a 3 dish delivery look-ahead. Analyzing the rollouts, we would expect that increasing the horizon to 4 would not significantly impact the reward for the two layouts used in Section 5.

F Near-optimal model-based planner

Given a fixed partner model, we implement a near-optimal model-based planner that plays with the fixed partner. The planner uses a two-layered A^* search in which:

1) on a low level we use A^* in the game state space with edges being basic player joint actions (one of which is obtained by querying the partner model). To reduce the complexity of such search, we remove stochasticity from the partner model by taking the argmax probability action.

2) on a higher level we use A^* search in the state space in which edges are high level actions, similarly to those of the near-optimal joint planner described in Appendix E. Unlike in that case, it is unfeasible to pre-compute all low-level motion plans, as each motion plan does not only depend on the beginning positions and orientations of the agents, but also on other features of the state (since they could influence the actions returned by the partner model).

G Planning experiments

Due to computational constrains, when evaluating planning methods (such as in Figure 5), we evaluated on a horizon of 100 timesteps, and then multiplied by 4 to make the environment horizon comparable to all other experiments. This is another source of possible suboptimality in the planning experiments.

When observing Figure 5, we see that P_{BC}+H_{Proxy} performs much worse than the red dotted line (representing planning with respect to the actual test model H_{Proxy}). This is due to the fact that in the planning experiments all agents are set to choose actions deterministically (in order reduce the

planning complexity – as mentioned in appendix F), leading the agents to get often stuck in loops that last the whole remaining part of the trajectory, leading to little or no reward.

H Human-AI experiments

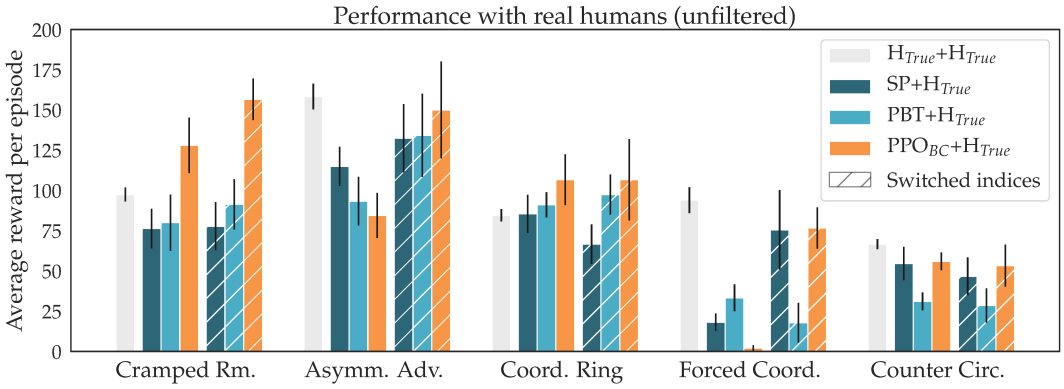


Figure 12: The results are mostly similar to those in Figure 6, with the exception of larger standard errors introduced by the non-cooperative trajectories. The reported standard errors are across the human participants for each agent type.

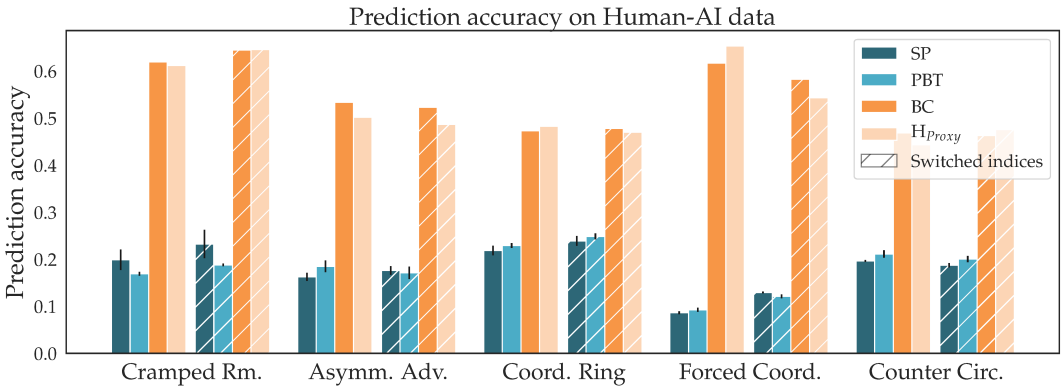


Figure 13: Accuracy of various models when used to predict human behavior in all of the human-AI trajectories. The standard errors for DRL are across the 5 training seeds, while for the human models we only use 1 seed. For each seed, we perform 100 evaluation runs.

The human-AI trajectories were collected with a horizon of 400 timesteps. The agents used in this experiment were trained with slightly different hyperparameters than those in the previous appendices: after we had results in simulation and from this user study, we improved the hyperparameters and updated the simulation results, but did not rerun the user study.

After manually inspecting the collected human-AI data, we removed all broken trajectories (human not performing any actions, and trajectories shorted than intended horizon). We also noticed that in a large amount of trajectories, humans were extremely non-cooperative, not trying to perform the task well, and mainly just observing the AI agent and interacting with it (e.g. getting in its way). Our hypothesis was that these participants were trying to "push the boundaries" of the AI agents and test where they would break – as they were told that they would be paired with AI agents. We also removed these non-cooperative trajectories before obtaining our results. Cumulatively across all layouts, we removed 15, 11, and 15 trajectories of humans paired with PBT, PPO_{SP}, and PPO_{BC} agents respectively.

In Figure 12 we report human-AI performance without these trajectories removed. Performing an ANOVA on all the data with agent type as a factor and layout and player index as a covariates showed a significant main effect for agent type on reward ($F(2, 250) = 4.10$, $p < .01$), and the post-hoc analysis with Tukey HSD corrections confirmed that PPO_{BC} performed significantly better than for PBT ($p < .01$), while fell just short of statistical significance for SP ($p = .06$).